FASE 6 · BANCO DE DADOS NA SOCIEDADE 5.0

# Smart HAS

## Do Firebase para o Oracle, com a regra dentro do banco

Nove tabelas modeladas, quatro functions e cinco procedures PL/SQL. O aplicativo pede um emprestimo e quem decide se ele pode acontecer e o proprio banco.

VÍDEO (YOUTUBE, NÃO LISTADO)
youtu.be/qZQMvaPy4I4

REPOSITÓRIO (GITHUB)
github.com/YuriRASousa/ComunidadeConectada_FASE5




**Yuri Raphael Alves de Sousa**

RM 556163 · Sistemas de Informação · FIAP 2026


Oracle PL/SQL · Spring Boot + JDBC · React Native · Angular



# O que mudou na solução

A troca de banco virou uma mudança de arquitetura: a regra desceu para o Oracle

### Antes (Fase 5)

- Firestore guardando documentos
- Senha no Firebase Authentication
- Login dependia de chamada externa
- HTTP puro e CORS liberado com curinga

### Agora (Fase 6)

- Nove tabelas relacionais no Oracle
- Senha com hash BCrypt na SH_USUARIO
- Login e uma consulta ao banco
- HTTPS por perfil e CORS fechado

### O que isso destravou

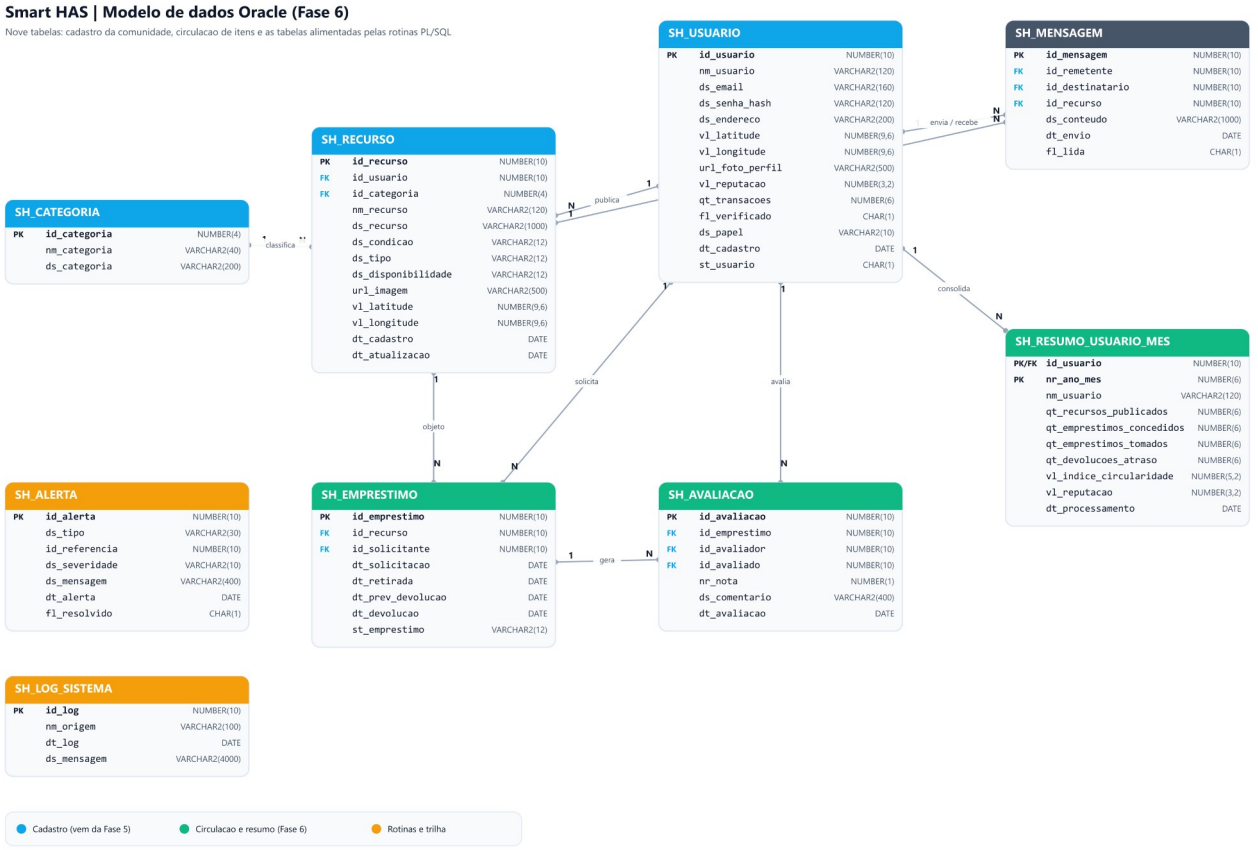
- Emprestimo com prazo e devolução
- Avaliação e reputação calculada
- Alertas de atraso automáticos
- Indicador de circularidade por mês

### O Firebase saiu por completo

Foram removidos o SDK firebase-admin, a configuração do Firebase, o cliente de autenticação e os tres repositorios Firestore. O token JWT emitido continua igual, entao o aplicativo React Native e o painel Angular nao precisaram de nenhuma alteração no contrato de autenticação: a troca aconteceu inteira do backend para dentro.

# Modelo lógico e físico no Oracle

Nove tabelas com prefixo SH_, oito sequences, seis indices e comentarios de dicionario




## Cadastro

- Categoria, usuario e recurso
- Mensagem entre moradores

## Circulação

- Emprestimo e avaliação
- Resumo mensal por morador

## Operação

- Alertas abertos pelas rotinas
- Trilha de execução em log

02 | PARTE 2

# O que o banco passou a garantir

Estados que o banco de documentos aceitava sem reclamar agora sao barrados na origem

### Integridade

- Chaves estrangeiras ligando recurso, dono, categoria e emprestimo
- E-mail unico por morador
- Chave composta no resumo: um morador por competencia

### Dominios com CHECK

- Tipo, condição e disponibilidade do item
- Status do emprestimo
- Nota de 1 a 5 e severidade do alerta

### Carga simulada

- 8 moradores, 20 itens, 15 emprestimos
- Datas relativas a SYSDATE
- Dois emprestimos vencidos de proposito, para a rotina ter o que achar

### Scripts na ordem de execução

`01_ddl` → `02_seed` → `03_log` → `04_functions` → `05_procedures` → `06_consultas`

# Functions

Uma devolve indicador, outra devolve dado formatado, as duas rodam dentro do SELECT

## FN_SH_INDICE_CIRCULARIDADE

- Emprestimos concedidos por item publicado no mes
- Valida a competencia AAAAMM e trata divisao por zero

## FN_SH_FICHA_RECURSO

- Ficha do item em uma linha pronta para a tela
- %ROWTYPE, NO_DATA_FOUND e TOO_MANY_ROWS tratados

## FN_SH_REPUTACAO

- Media das notas recebidas nas devoluções
- Morador sem avaliação comeca em 5

## FN_SH_DISTANCIA_KM

- Haversine entre duas coordenadas
- DETERMINISTIC: ordena a vitrine por proximidade

```
SELECT nm_usuario,
       fn_sh_reputacao(id_usuario)                    AS reputacao,
       fn_sh_indice_circularidade(id_usuario, 202609) AS circularidade
  FROM sh_usuario WHERE st_usuario = 'A' ORDER BY 3 DESC;
```

# Procedures

Duas atendem o usuario final, duas sao rotinas de operação

## REGISTRA_EMPRESTIMO

- Parametros IN e OUT
- Recusa dono pedindo o proprio item, item indisponivel, tipo errado e usuario inativo
- COMMIT no sucesso, ROLLBACK no erro

## REGISTRA_DEVOLUCAO

- Avaliação opcional de 1 a 5
- Abre alerta quando a devolução atrasa
- Atualiza a reputação usando a function

## DETECTA_ATRASOS

- CURSOR FOR LOOP nos emprestimos vencidos
- IF define a severidade pelo tamanho do atraso
- Nao repete alerta ja aberto

## GERA_RESUMO_MENSAL

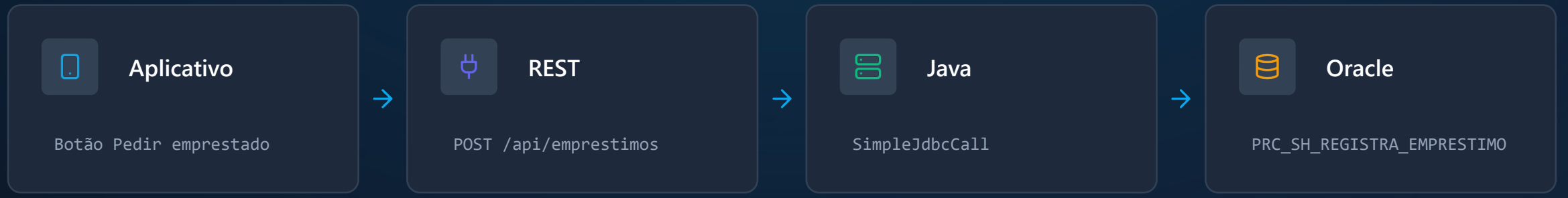
- BULK COLLECT e FORALL sobre TABLE OF ...%ROWTYPE
- Recarrega SH_RESUMO_USUARIO_MES para a competencia
- O painel le numero pronto em vez de refazer a conta

## GRAVA_LOG e ATUALIZA_REPUTACAO

- PRAGMA AUTONOMOUS_TRANSACTION: o log sobrevive ao ROLLBACK
- Toda rotina registra inicio, fim e erro
- Reputação gravada a partir da function

# Do toque no aplicativo ate a procedure

O requisito de acionar PL/SQL pelo backend Java, no fluxo mais usado do produto




```
new SimpleJdbcCall(jdbc)
    .withProcedureName("PRC_SH_REGISTRA_EMPRESTIMO")
    .withoutProcedureColumnMetaDataAccess()
    .declareParameters(
        new SqlParameter("p_id_recurso", Types.NUMERIC),
        new SqlOutParameter("p_ds_retorno", Types.VARCHAR));
```

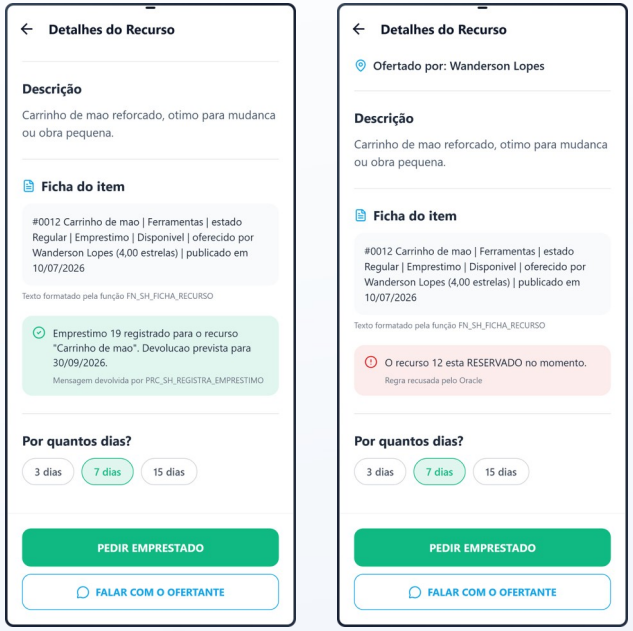
### Erro que vira resposta HTTP

- RAISE_APPLICATION_ERROR entre -20000 e -20999
- O backend le so a primeira linha e descarta a pilha
- 404, 409 ou 400 conforme o codigo

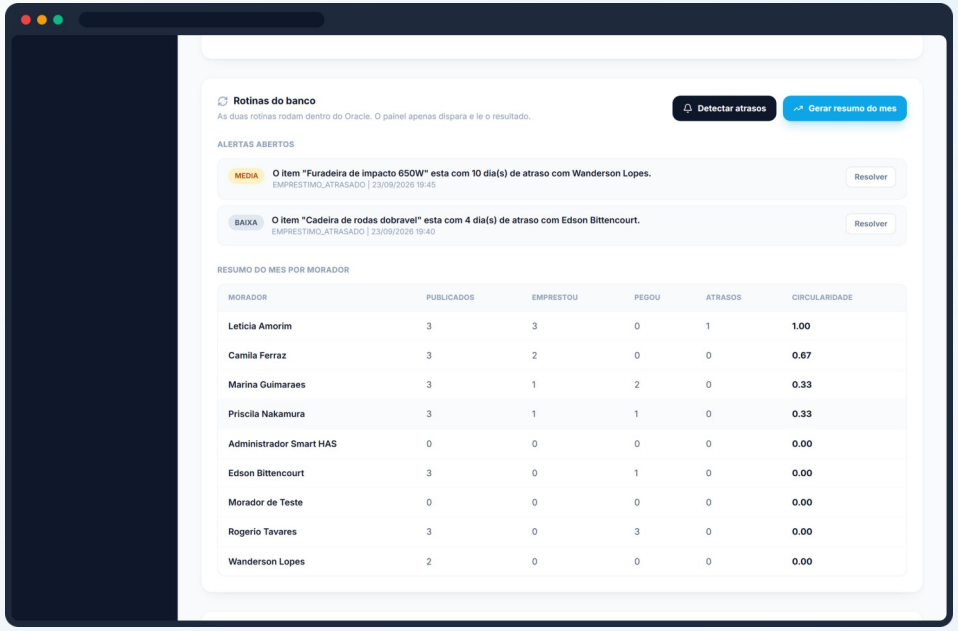
O morador tenta pegar emprestado o proprio item e le na tela: "O solicitante e o proprio dono do recurso 7." Esse texto foi escrito dentro da procedure, nao no aplicativo.

# A mesma procedure no aplicativo e no painel

Telas capturadas com o backend ligado no Oracle da FIAP




Aceito e recusado pelo Oracle




Painel: rotinas, alertas e resumo do mês

## No aplicativo

- Ficha do item, pedido com prazo e devolução

## Regra recusada

- O texto do erro foi escrito na procedure

## No painel

- Dois botoes disparam as rotinas de operação

FECHO

# O que fica desta fase

## Entregue

- Modelo relacional completo, com carga simulada
- Quatro functions e cinco procedures
- Backend Java chamando procedure por JDBC
- Transporte em TLS 1.3 e CORS restrito
- Aplicativo e painel usando as duas pontas

## Proximos passos

- Agendar as rotinas com DBMS_SCHEDULER
- Agrupar as rotinas em um package
- Historico da tabela de resumo virando grafico no painel

A regra de quem pode pegar um item emprestado saiu do Java e foi morar junto dos dados. Agora ela vale para o aplicativo, para o painel e para qualquer carga feita direto no banco.

VÍDEO (YOUTUBE, NÃO LISTADO)
youtu.be/qZQMvaPy4I4

REPOSITÓRIO (GITHUB)
github.com/YuriRASousa/ComunidadeConectada_FASE5



**Yuri Raphael Alves de Sousa**
RM 556163